Fresh Solutions: a reproducible adapter harness for applied metaheuristic benchmarks

UBC FRESH Lab

November 3, 2026

Abstract

Applying metaheuristics to hard combinatorial problems is often slowed less by the choice of metaheuristic than by the engineering around it: problem-specific move representations, exact incremental state, replayable evidence, stopping rules, and benchmark bookkeeping are usually rebuilt for each project. Fresh Solutions provides an adapter contract and a reproducible benchmark harness, together with two reference implementations: quadratic-assignment robust tabu search and travelling-salesman iterated Lin–Kernighan. We evaluate the references on QAPLIB and TSPLIB and report the full distribution, including failures. We also run a framework-use check: the QAP tabu trajectory is reproduced with the incumbent held in the engine's resident-state contract. On 40 runs across four QAPLIB instances, all runs stopped on the same 100,000-iteration plateau rule under a 1,000,000 iteration cap, all framework objectives matched the standalone and production objectives exactly, and all incumbents replayed exactly. The framework path cost 1.24 times the CPU time of the private-array delta path (95% CI 1.17–1.31). A 10,000-kick TSP campaign with 1,000-kick patience plateau-stopped on all 20 runs and reached all 20 known optima. The contribution is the harness, exact replay contract, reference adapters, and this bounded framework-use result, not a state-of-the-art solver claim.

1 Introduction

Operations-research practitioners rarely need a new metaheuristic. They need a reliable way to apply known methods to a new problem, preserve exact feasibility and objective accounting, and produce evidence that another person can replay. That work is repetitive and easy to get subtly wrong. Problem-specific move representations, incremental state, stopping rules, benchmark harnesses, seed handling, and evidence records are rebuilt for each project.

Fresh Solutions is a response to that engineering problem. It provides a problem-adapter contract, exact resident state and reversible moves, local and population search paths, checkpointing, exact replay, convergence records, and public-library reference implementations for QAP and TSP. The paper asks a narrow engineering question: can a user add a problem-specific neighbourhood or reference implementation and obtain credible, replayable evidence without rebuilding the surrounding machinery? We answer it with a bounded framework-use experiment: a QAP tabu path whose incumbent is held in the engine's resident state reproduces the private-array delta-matrix trajectory exactly, at a measured state-management cost. We do not claim that the generic simulated-annealing layer is competitive with the adapter-specific references.

The paper makes four contributions. First, it specifies a small adapter contract that separates a problem's exact state, moves, and feasibility from the search layer. Second, it specifies a replay and evidence protocol: an incumbent is not a result unless a fresh adapter evaluation reproduces the stored objective and the record names the instance, seed, budget, stopping reason, and provenance.

Third, it provides two public reference implementations, robust tabu for QAP and iterated Lin–Kernighan for TSP, and reports their full distributions, including tails and cap hits. Fourth, it reports a convergence-controlled framework-use check for QAP state management and a separate TSP convergence campaign; both are bounded empirical results, not a general-framework claim.

The scope is deliberately narrow. The QAP convergence panel has four instances; the TSP convergence panel has four; the broad panels screen many more instances but do not claim per-run convergence. The external LKH binary is a capability reference and is not bundled. The generic population and simulated-annealing layers are implemented and tested, but the headline results come from the adapter-specific reference modules. This is the honest boundary of the current evidence.

The remainder of the paper is organized as follows. Section 2 reviews the problem and benchmarking literature. Section 3 describes the adapter, resident state, search layers, and evidence protocol. Section 5 reports the QAP evidence and the framework-use experiment. Section 6 reports the TSP convergence campaign and the LKH boundary. Section 7 describes the applied case, and Section 8 discusses limitations and the claim boundary.

2 Related work

2.1 Quadratic assignment

The quadratic assignment problem assigns facilities to locations while minimising a flow–distance product [Koopmans and Beckmann, 1957]. It is NP-hard [Sahni and Gonzalez, 1976] and remains a standard test bed for metaheuristics because small changes in the flow and distance matrices can produce very different search landscapes [Loiola et al., 2007]. QAPLIB is the common public benchmark library [Burkard et al., 1997]. Exact methods are limited to modest sizes or special structure, so the literature is dominated by local search and hybrid methods.

Robust tabu search is a long-standing baseline [Taillard, 1991]. It scans facility swaps, keeps a tabu tenure for recently changed pairs, and accepts the best non-tabu move even when it worsens the objective. The reactive variant adapts the tenure to the search history [Battiti and Tecchiolli, 1994]. More recent breakout local search and memetic methods combine tabu or descent with perturbation and population recombination [Benlic and Hao, 2013]. Taillard's comparison of iterative searches remains a useful reminder that restart, perturbation, and acceptance rules can matter as much as the neighbourhood [Taillard, 1995]. Our QAP adapter follows the robust-tabu move-and-memory idea, with exact swap deltas and an optional delta-matrix path; it is a reference implementation, not a claim of state-of-the-art QAP performance.

2.2 Traveling salesman

The traveling salesman problem has an equally long heuristic literature. The Lin–Kernighan variable-depth exchange [Lin and Kernighan, 1973] remains the foundation of the strongest local-search methods. Iterated local search adds perturbation to a local optimum and is effective when the perturbation is matched to the neighbourhood [Lourenço et al., 2003]. Helsgaun's LKH implementation [Helsgaun, 2000] and the Concorde branch-and-cut code [Applegate et al., 2006] are the standard strong references. TSPLIB provides the public instances [Reinelt, 1991], and the local-search case study of Johnson and McGeoch documents the gap between a textbook 2-opt implementation and a carefully engineered candidate-list method [Johnson and McGeoch, 1997]. We implement a bounded candidate-list LK-style exchange and use external LKH only as an optional

capability reference, not as part of the package. Our contribution is the adapter and evidence boundary, not a new TSP heuristic.

2.3 Benchmarking and reproducibility

Computational experiments with heuristics are easy to make incomparable. Hooker argued that algorithm comparisons should test hypotheses about why a method works, not just report wins [Hooker, 1995]. Barr et al. proposed a reporting discipline for heuristic experiments [Barr et al., 1995], and McGeoch developed an experimental method for algorithm simulation [McGeoch, 1996]. Beiranvand et al. collect best practices for comparing optimization algorithms, including budget accounting, multiple seeds, and distributional reporting [Beiranvand et al., 2017]. COCO provides a platform for comparing continuous optimizers [Hansen et al., 2020], and irace provides automatic algorithm configuration with racing [López-Ibáñez et al., 2016]. These systems are stronger than our harness in their statistical machinery; our focus is the problem-adapter and exact-replay boundary for discrete, domain-specific models.

Reproducibility in applied optimisation depends on more than a seed. It depends on the compiled model, candidate order, objective weights, hardware, and replay procedure. The general reproducible research literature makes the same point for computational science [Peng, 2011, Sandve et al., 2013, Wilson et al., 2014, Stodden et al., 2016, Donoho et al., 2009]. Containers and environment capture reduce the software part of the problem [Boettiger, 2015, Kurtzer et al., 2017]. Our harness stores the problem instance identifier, pinned solver commit, seed, budget, stop reason, exact-replay result, and hardware with each record. The design is deliberately closer to a benchmark harness than to a new metaheuristic.

2.4 Software frameworks

General metaheuristic frameworks usually expose a solution representation, an objective, and a neighbourhood, then supply search operators [Blum and Roli, 2003, Hoos and Stützle, 2004]. The engineering burden shifts to the adapter, but the evidence boundary is often left to the user. Benchmarking platforms such as COCO and configuration tools such as irace solve adjacent problems: COCO standardises the comparison of continuous optimizers, and irace standardises the tuning of algorithm parameters [Hansen et al., 2020, López-Ibáñez et al., 2016]. Neither is a drop-in answer for a compiled spatial model with domain-specific feasibility and exact replay.

Our adapter contract makes the evidence boundary part of the interface: exact evaluation, reversible moves, replay, and convergence records are required operations rather than optional utilities. That choice is the difference between a library of operators and a reproducible benchmark harness. It also explains the paper's narrow claim. A framework that owns the evidence boundary can be evaluated on whether the boundary is preserved; a framework that owns the search algorithm must be evaluated on solution quality against the state of the art. This paper tests the first claim, not the second.

The design is closer in spirit to a laboratory notebook than to a solver. The notebook does not make the experiment correct by itself, but it makes the experiment inspectable. The adapter is the page format; the evidence record is the entry; the replay gate is the check that the entry still reproduces. That metaphor is not a substitute for algorithmic novelty, but it is a useful way to describe the contribution without inflating it.

3 Framework

3.1 Design goals

The harness is built around four goals. The first is *exactness*: every reported assignment is a complete discrete assignment, and every objective is computed by the problem adapter, not by a search heuristic's internal counter. The second is *reversibility*: a move can be applied and undone without rebuilding the whole assignment, so a local search can test neighbours without allocating a new state for every candidate. The third is *replayability*: a result is stored with enough metadata that a third party can re-evaluate the incumbent and recover the same objective and feasibility. The fourth is *separation*: the search layer sees only the adapter interface, so a domain-specific neighbourhood can be added without changing the engine.

These goals are deliberately modest. The harness does not attempt to infer a problem's structure, choose a neighbourhood automatically, or prove optimality. Those are research contributions in their own right. The harness is the scaffolding that makes such contributions comparable and replayable.

3.2 Adapter contract

An adapter supplies discrete variables, their options, a complete initial and random assignment, exact and batched evaluation, feasibility, mutation and crossover, and reversible moves. The engine sees only this interface. An assignment is a mapping from every variable to one option; for a permutation problem, the adapter is responsible for preserving the permutation invariant. A change records the variable, the option before, the option after, and an optional move kind and segment size. The adapter's `apply` and `undo` operations must be exact inverses on the assignment.

The contract is intentionally smaller than a general modelling language. It does not expose linear constraints, gradients, or symbolic expressions. That restriction keeps the adapter implementable for compiled spatial models and public combinatorial libraries, where the objective is often a domain-specific calculation rather than a closed-form expression. A model that cannot supply exact evaluation and reversible moves is outside the contract; the harness does not pretend to optimise it.

The assignment representation is a mapping from integer variable positions to integer options. For a permutation problem, the options are the same integer set as the positions, and the adapter validates that the mapping is a bijection. For a spatial model, a variable may be a block and an option may be a treatment or a harvest period; the adapter translates the integer option into the domain value. The engine never needs to know which interpretation is correct. This is why the same resident-state and replay code can serve QAP, TSP, and the applied spatial case.

The adapter also supplies `evaluate_batch`, which is used by the population layer and by external baseline adapters. The batch path is not required to be vectorised; it is a contract for returning exact evaluations, not a performance promise. A vectorised adapter can override it; a simple adapter can loop over `evaluate`. The distinction keeps the contract usable for both small public instances and compiled models.

3.3 Exact state and replay

Every reported incumbent is a complete assignment. Evaluation is exact, and the stored assignment must re-evaluate to the stored objective. The benchmark scripts do not trust the optimizer's internal counter; they replay the result through the adapter. A record with a mismatched assignment hash or replayed objective is not evidence.

Replay is more than a checksum. The record names the public instance and its known optimum or best-known value, the pinned solver version or commit, the seed, the iteration or kick budget, the

stopping reason, the exact-replay result, and the wall and CPU time. A safety-cap hit is recorded as a cap hit; it is not relabelled as convergence. The replay gate runs the stored assignment through a fresh adapter state, so a record cannot pass by reusing the optimizer's cached objective.

The QAP and TSP adapters both validate the assignment before evaluation. QAP checks that the assignment is a permutation of facilities; TSP checks that it is a permutation of cities. Both return an objective and a feasibility flag. A malformed assignment raises an error rather than producing a number. This matters because a permutation repair inside an optimizer can hide an invalid move; the adapter boundary makes the repair explicit.

3.4 Framework-managed state

The engine exposes `ResidentAssignment`, which holds one complete assignment and routes every accepted move through the adapter's reversible `apply/undo` operations, followed by an exact `evaluate`. The framework-use experiment repeats the QAP tabu scan and tabu memory from the reference path, but holds the incumbent in this resident state. The private-array path and the framework path use the same identity start, seed, delta matrix, tenure rule, patience, and iteration cap. Execution order is alternated by seed to balance host load; both wall and process CPU time are recorded.

3.5 Search layers

The generic local layer is a simulated annealing search over adapter proposals. It evaluates a proposed move through the resident state, accepts improving moves, and accepts worsening moves with a temperature-dependent probability. The generic population layer supplies mutation, crossover, and elitism. The hybrid layer hands a population elite to exact local refinement. These layers are useful for smoke tests and for new problems that do not yet have a specialised neighbourhood, but they are not the source of the strongest results in this paper.

The reference implementations are separate adapter-specific modules. The QAP module uses robust tabu search with an exact delta matrix; the TSP module uses a candidate-list Lin–Kernighan exchange with double-bridge kicks. The engine still owns the assignment and the replay boundary, but the neighbourhood and acceptance logic are problem-specific. This is the correct division for a benchmark harness: a generic engine cannot contain every competitive neighbourhood, and a domain module should not have to reimplement evidence handling.

3.6 Evidence protocol

A reportable result names the pinned solver version or commit, the public instance and known or best-known value, the seed and budget, the stopping reason, the exact-replay result, and the wall and CPU time. Publication-facing campaigns use a patience rule; safety-cap hits are recorded as non-converged.

The evidence format is JSON. Each campaign has per-run records and a summary. The per-run record keeps the assignment and the search counters; the summary keeps the distribution. The repository also keeps the runner, the replay script, the instance files or their provenance, and the environment. A reader can therefore rerun the campaign, replay the stored incumbents, or compare the reported distribution without reading the paper prose.

A minimal record contains the instance name, adapter name, seed, budget, stop reason, objective, feasibility, assignment, exact-replay result, wall time, CPU time, and a metadata block with the solver commit and hardware. The record does not contain a confidence interval or a performance profile; those are summary statistics. The summary contains the per-instance distribution and

the converged/cap-hit counts. The separation is deliberate: a per-run record is evidence, while a summary is an interpretation of evidence.

The replay gate is intentionally stricter than the optimizer. It does not call the search code. It constructs a fresh adapter, evaluates the stored assignment, and compares the objective and feasibility. For a permutation problem it also validates the mapping. A record that passes the gate is reproducible in the narrow sense used here; a record that fails is a defect even if its objective looks plausible.

3.7 Implementation and provenance

The harness is implemented in Python and uses NumPy [Harris et al., 2020] for array operations and SciPy [Virtanen et al., 2020] for an independent QAP baseline. The optional accelerated QAP path uses Numba [Lam et al., 2015]; the reference path remains available without it. The public instances are QAPLIB [Burkard et al., 1997] and TSPLIB [Reinelt, 1991]. The TSA24 applied case is a public UBC-FRESH artifact [UBC FRESH Lab, 2026]. External LKH is a separate research-use binary [Helsgaun, 2019]; it is not bundled and its results are reported as a capability boundary, not as part of the package.

The campaigns record the hardware and software environment with the evidence. The published tables use the pinned public solver version and the public instance files. The evidence release itself is a candidate until the maintainer approves a tag; the paper does not treat an unapproved candidate as a published artifact.

3.8 Artifact structure

An evidence directory contains the per-run JSON, the summary JSON, the replay script, and the instance provenance. The QAP framework directory also contains the paired-run runner and the sensitivity comparison. The TSP directory contains the convergence runner, the 2k and 10k campaigns, and the replay script. The analysis scripts read the JSON and print the LaTeX table rows, so a reviewer can regenerate a table without copying numbers from the paper.

The artifact is designed to fail closed. A replay script compares the stored objective with a fresh adapter evaluation and raises on a mismatch. A runner records a safety-cap hit rather than silently dropping the run. A summary records the number of converged runs, not only the mean of the successful ones. The manifest stores SHA-256 hashes for the evidence files; a tag is not created until the maintainer approves the candidate.

The public QAPLIB and TSPLIB instances are not redistributed in a modified form. The TSA24 artifact is published separately with its compiled data and licence statement. External LKH is not bundled; the provenance sidecar records its version and command, and the paper treats it as an external capability reference. This separation matters for licensing: the Fresh Solutions code and documentation are MIT-licensed, derived artifacts use CC BY 4.0, and the BC source data carries OGL-BC attribution.

3.9 Failure accounting

The harness distinguishes four outcomes: plateau stop, safety-cap hit, budget exhaustion, and invalid replay. A plateau stop is a run that met its declared patience rule. A safety-cap hit is a run that reached a wall-clock cap before its patience rule; it is not convergence evidence. Budget exhaustion is a run that reached its iteration or kick cap; it is retained but not called converged. Invalid replay is a record that cannot reproduce its objective; it is removed from the evidence and reported as a defect, not averaged into a table.

Table 1: Campaigns and stopping rules. The broad panels are fixed-budget screens and are not pooled with the convergence panels.

Campaign	Panel	Seeds	Budget	Stop rule
QAP convergence	4 QAPLIB	10	1,000,000 it.	100,000 patience
QAP framework	4 QAPLIB	10	same as above	same as above
Large QAP	4 QAPLIB	5 starts	1,000,000 it.	10,000 patience
Broad QAP	134 QAPLIB	5	600 s cap	10,000 patience
TSP convergence	4 TSPLIB	5	10,000 kicks	1,000 patience
TSP 2k check	4 TSPLIB	5	2,000 kicks	200 patience
TSP 20-kick pilot	4 TSPLIB	5	20 kicks	none
Broad TSP	78 TSPLIB	5	600 s cap	100 patience
Applied TSA24	1 instance	5	100,000 gen.	fixed budget

This taxonomy is simple, but it prevents a common failure mode in heuristic papers: reporting the converged subset while omitting the runs that hit the cap. The QAP broad panel reports 552 of 670 patience stops and 318 optima; the TSP broad panel reports 236 of 390 patience stops and 183 optima. The denominator is visible, and the cap hits are not hidden in a footnote.

4 Experimental design

4.1 Protocols and stopping rules

Table 1 summarizes the campaigns and their stopping rules. The convergence panels are the evidence for per-run plateau statements; the broad panels are screens. The table is part of the protocol, not a result: it fixes the denominator before the reader sees a mean.

The QAP convergence panel uses four QAPLIB instances: `tai12a`, `tai20a`, `tai25a`, and `tai30a`. Each run starts from the adapter's identity assignment, uses a 100,000-iteration patience rule, and has a 1,000,000-iteration cap. A run is reported as plateau-stopped only when the stop reason is `patience`; a cap hit is reported as non-converged. The panel uses ten seeds per instance.

The framework comparison runs two implementations of the same tabu trajectory. The standalone path keeps the incumbent in a private array and uses Taillard's delta matrix. The framework path keeps the incumbent in `ResidentAssignment`, applies every accepted swap through the adapter, and calls the adapter's exact evaluator. Both use the same start, seed, delta matrix, tenure, patience, and cap. The order of the two paths is alternated by seed to reduce host-load bias; both wall time and process CPU time are recorded. The production accelerated path is run as a third check, but its wall time is not used for the overhead estimate.

The final QAP campaign runs one paired seed per worker process. The pool uses the `spawn` start method so that the optional Numba path is not inherited across a fork. Each worker runs the standalone path, the framework path, and the production path in sequence; the paired ratio is therefore computed inside one worker on one host, not across different machines. The worker count is recorded because it affects wall time; the CPU ratio is less sensitive to the number of workers.

The TSP campaign uses the same process-pool design, one seed per worker. The wall time in the TSP table includes the candidate-list construction and the LK passes; it is not a pure kick count. The 2,000-kick and 10,000-kick campaigns use the same starts and seeds, so the longer campaign is a budget extension of the same experiment, not a different instance set.

The TSP convergence panel uses `berlin52`, `kroA100`, `kroB100`, and `ch150`. Each run starts from the corresponding converged Or-opt tour and uses 10,000 double-bridge kicks with a 1,000-kick

patience rule. Five seeds are used. The 2,000-kick/200-patience and 20-kick campaigns are retained as shorter-budget checks and are not pooled with the final table.

The broad QAP and TSP panels are separate fixed-budget screens. They use the public QAPLIB and TSPLIB collections, five seeds, and a 600-second safety cap where stated. A broad-panel run that hits the cap is reported separately; it is not counted as a converged result. The large-QAP multi-start experiment uses five random starts per instance. SciPy's quadratic assignment routine [Virtanen et al., 2020] is run as a weak, fast external baseline, not as a matched-quality baseline.

4.2 Instance selection

The QAP panel spans $n = 12$ to $n = 30$ and uses instances with published optima or best-known values: `tai12a`, `tai20a`, `tai25a`, and `tai30a`. The size range is driven by the framework comparison: both the private-array and resident-state paths are pure Python at the trajectory level, and a larger QAP would make the paired comparison a measurement of memory bandwidth rather than state-management cost. The large-QAP table is run separately with the accelerated path.

The TSP panel spans $n = 52$ to $n = 150$ and uses four TSPLIB instances with known optima. The starts are the converged Or-opt tours recorded in the evidence, so the comparison measures the LK and kick layers from a common starting point. The broad TSP panel covers 78 instances with $n \leq 1000$; it uses a different start generator and a fixed budget, so it is reported separately.

The broad QAP panel parses 134 QAPLIB instances; 25 `esc8` files are excluded because the parser rejects their incomplete format. The parser errors are recorded as parse errors, not as solver failures. This is a small example of the provenance discipline: a missing benchmark is not a loss.

4.3 Statistics and provenance

The summary of each panel reports the mean, median, best, and worst gap, the number of plateau stops and cap hits, and the wall and CPU times. The framework overhead is the paired ratio of framework to standalone CPU time; the paper reports the mean and a percentile bootstrap 95% confidence interval over seeds. Because the framework and standalone objectives are exactly equal on every seed, a significance test for a quality difference would be inappropriate.

The paper does not use a performance profile across unlike instance families. A QAP gap normalized by the known optimum and a TSP gap normalized by the tour length are not on a common scale, and a profile that mixed them would create a false ranking. The tables therefore keep the families separate and report the distribution within each family. The same rule applies to time: wall time under a parallel campaign is descriptive, and the CPU ratio is used only for the paired framework comparison.

Where a table reports a mean, it also reports the worst case or the full denominator. This is a deliberate response to the heavy tails in the broad panels. A mean without a tail can make a method look more reliable than it is; a best-of-five without a mean can make a lucky run look like a portfolio.

The final QAP and TSP campaigns run on a Linux container with 72 logical CPUs and 256 GB of RAM. The QAP final campaign uses 40 worker processes; the TSP final campaign uses 20. The evidence records the platform, processor string, Python version, worker count, solver commit, instance files, and stopping configuration. The worker count is part of the provenance because wall times under parallel load are descriptive, not a single-thread benchmark.

4.4 Threats to validity

The first threat is panel size. Four QAP and four TSP instances are enough to test a framework path and a convergence protocol, but not enough to estimate performance across the full instance distributions. The broad panels address coverage, but their fixed budgets mean they cannot substitute for the convergence panels.

The second threat is baseline strength. The QAP reference is robust tabu, a standard but not state-of-the-art method; SciPy FAQ is weak. The TSP reference is a bounded LK implementation, and external LKH is stronger on the large tail. The paper therefore reports gaps to known optima and a capability boundary, not a claim that Fresh Solutions dominates published solvers.

The third threat is timing. The framework comparison is CPU-bound and run in parallel. Alternating order reduces, but does not eliminate, host-load bias. Process CPU time is the primary overhead statistic; wall time is reported for operational context.

The fourth threat is stopping. Patience is a plateau rule, not a proof of global optimality. A run can stop on patience and still be far from the optimum, especially on a deceptive landscape. The paper uses "plateau-stopped" or "converged for this evidence" rather than "converged" without qualification.

5 QAP evidence

The QAP reference implementation uses robust tabu search over facility swaps. The objective is

$$\min_{\pi \in \Pi_n} \sum_{i=1}^{n} \sum_{j=1}^{n} F_{ij} D_{\pi(i)\pi(j)}, \tag{1}$$

where F is the flow matrix and D the distance matrix. The adapter uses exact swap deltas; the accelerated path uses Taillard's delta-matrix update for symmetric zero-diagonal instances and a Numba general scan for asymmetric instances.

The tabu search scans every facility swap at each iteration and chooses the best non-tabu move. A move is tabu while its tenure counter is active, unless it would improve the best-known objective; that aspiration rule prevents the tabu list from blocking a new incumbent. The tenure is sampled uniformly between $n/2$ and $3n/2$ for a problem of size n. The tabu memory is an $n \times n$ matrix, which is small for the panel sizes but becomes a real memory cost for large instances. The search does not use a long-term frequency memory or a path-relinking layer; it is the robust-tabu baseline, not a hybrid.

For a candidate swap, the exact objective delta is the sum of the flow–distance contributions of the two facilities against every other facility, with the diagonal and cross terms handled explicitly. The adapter computes that delta directly in the reference path. The accelerated path stores the gain for every pair and updates the matrix after the accepted swap, using Taillard's recurrence for symmetric zero-diagonal instances. After a swap, the delta matrix is updated in $O(n^2)$ time; the scan itself is also $O(n^2)$, so one iteration costs $O(n^2)$ for the accelerated symmetric case. The unaccelerated reference scan computes each swap delta directly and is used as a correctness fallback.

The tabu search stops on a patience rule or a hard budget. On a plateau stop, the incumbent is the best assignment seen, not necessarily the current assignment. The best assignment is stored as a complete mapping, so the replay gate does not need to reconstruct the move history. This is a small design choice with a large evidence benefit: the record is independent of the tabu memory and the search counters.

The framework path shares the scan, the tabu memory, and the tenure rule. The difference is the incumbent representation: the framework path calls `apply` for the accepted swap and `evaluate`

Table 2: QAP robust tabu, ten seeds, 100,000-iteration patience and 1,000,000-iteration cap. All 40 runs plateau-stopped and replayed exactly. The CPU ratio is the framework-managed path divided by the private-array delta path.

Instance	Mean gap	Median gap	Worst gap	Mean iter.	CPU ratio	Patience
`tai12a`	0.000%	0.000%	0.000%	100,186	1.327	yes
`tai20a`	0.030%	0.000%	0.304%	147,748	1.170	yes
`tai25a`	0.077%	0.000%	0.402%	175,611	1.181	yes
`tai30a`	0.245%	0.391%	0.491%	161,046	1.279	yes

for the resulting assignment, while the standalone path mutates a private array and updates the objective from the delta. The two paths therefore test the cost of routing state through the adapter, not a different search strategy.

5.1 Convergence-controlled QAP panel

Table 2 reports the final QAP panel: ten seeds per instance, a 100,000-iteration plateau rule, and a 1,000,000-iteration cap. Every one of the 40 runs stopped on `patience`; no run hit the cap. The framework path matched the standalone and production objectives on every seed, and every incumbent replayed exactly through the adapter.

The distribution matters. `tai12a` reached the optimum on all ten seeds; `tai20a` reached it on nine, `tai25a` on eight, and `tai30a` on four. The remaining `tai30a` runs clustered around the 0.391% plateau, with one at 0.491%. The mean is therefore not a claim of uniform success: it is the average of a distribution with a persistent gap on the hardest instance.

The fixed-budget table used in the earlier draft was not a per-run convergence table. At 10,000 fixed iterations its means were 0.000%, 0.403%, 0.685%, and 0.792%; the plateau-stopped panel improves the last three to 0.030%, 0.077%, and 0.245%. The difference is a budget effect, not a claim that the longer runs prove global optimality. The overall framework CPU overhead is 1.239 (95% bootstrap CI 1.168–1.310); per-instance ratios range from 1.170 to 1.327. The largest overhead is on the smallest instance, where fixed per-iteration state bookkeeping is a larger share of a short run; the larger instances also pay for the exact evaluation but amortise the setup cost over more work.

5.2 Broad panel and large instances

The broad panel below is a separate fixed-budget screen, not the convergence panel above. Across 134 parseable QAPLIB instances and five seeds, 552 of 670 runs stopped on patience and 318 hit the known optimum. The converged subset has median gap 0.000% and mean 1.823%, with a worst case of 36.4%. The mean is pulled by a small number of hard outliers; the median is not evidence that the tail is solved.

Table 3 reports the large-instance multi-start results. The best-of-five values are not a portfolio-improvement claim; the mean values show that the multi-start result is better on `tai256c` and `tho150` but worse on `esc128` and `tai150b`. SciPy FAQ is a weak, fast external baseline; it wins `tai150b` and loses the other three on mean gap.

The large-instance table is a mixed result, not a portfolio win. Fresh tabu is better on `esc128` and `tai256c`, worse on `tai150b`, and only modestly better on `tho150`. The SciPy baseline is fast and, on `tai150b`, better than the tabu path; on `tai256c` it is far worse. A best-of-five column would hide the tail, so the table reports both the best and mean. All twenty large-instance incumbents

Table 3: Large QAPLIB instances, five random starts, all stopping on patience. SciPy FAQ is an independent external baseline with the same five-seed protocol.

Instance	Fresh best	Fresh mean	SciPy mean	Fresh worst
`esc128`	0.000%	1.250%	12.500%	3.125%
`tai150b`	2.008%	2.976%	1.861%	3.699%
`tai256c`	0.350%	0.413%	120.481%	0.509%
`tho150`	0.166%	0.507%	1.523%	0.902%

replay exactly. The honest summary is that the reference tabu implementation is competitive on some large instances and not on others, and that the harness reports both outcomes.

6 TSP evidence

The TSP reference implementation uses two layers: Or-opt/2-opt for construction and a bounded candidate-list Lin–Kernighan-style exchange with deterministic double-bridge kicks. For a tour π, the objective is

$$L(\pi) = \sum_{i=1}^{n} d_{\pi(i),\pi(i+1)}, \qquad \pi(n+1) = \pi(1). \tag{2}$$

The construction layer uses nearest-neighbour starts plus 2-opt and Or-opt exchanges. The Lin–Kernighan layer keeps a candidate list of the 16 nearest neighbours per city, grows a sequential exchange to depth four, and accepts a tour only when the complete exchange is improving. The candidate list bounds the branching factor; the depth bound keeps the bookkeeping finite. A double-bridge perturbation chooses cuts $a < b < c < d$ and reconnects the segments in the order $[0, a) \cup [c, d) \cup [b, c) \cup [a, b) \cup [d, n)$, which changes four edges without reversing a long segment. The perturbed tour is passed back to the LK exchange. The process continues until a kick budget or a patience rule stops it.

The Or-opt/2-opt construction is deterministic for a fixed start. The initial tours are the converged Or-opt tours from the earlier campaign, so the TSP table measures the LK and kick layers rather than the construction layer. This is a controlled comparison, not a claim that the construction is irrelevant. A different start would change the absolute gap and the number of kicks needed to reach the optimum.

The implementation is a bounded LK-style exchange, not the full LKH method. It does not use α-nearness, fractional cascading, or the full LKH gain criteria. That is why the broad panel shows a large-instance tail and why external LKH is reported separately. The convergence panel is chosen to test the evidence protocol and the perturbation schedule, not to claim parity with LKH.

6.1 Convergence-controlled iterated Lin–Kernighan

Table 4 reports the final convergence campaign: 10,000 double-bridge kicks with a 1,000-kick plateau rule, five seeds, starting from the converged Or-opt tours. All 20 runs stopped on `patience`, all 20 tours replayed exactly, and all 20 reached the known optimum. The 2,000-kick and 20-kick runs are retained as shorter-budget checks; they are not the convergence table.

A 2,000-kick/200-patience pilot reached 19 of 20 optima and a 0.077% worst gap; the 20-kick pilot reached three optima and a 0.383% mean gap on `ch150`. The kicks ablation still separates the effect of the perturbation: LK-only means 0.780–2.580%, ILK-5 means 0.000–0.558%, and ILK-20 means 0.000–0.383%. The double-bridge kicks are material, not a cosmetic restart.

Table 4: TSP iterated Lin–Kernighan from converged Or-opt tours: five seeds, 10,000-kick cap, 1,000-kick patience. All 20 runs plateau-stopped, reached the known optimum, and replayed exactly.

Instance	Or-opt mean gap	ILK mean gap	ILK worst gap	Plateau	Mean wall
`berlin52`	6.433%	0.000%	0.000%	5/5	754.3 s
`kroA100`	7.323%	0.000%	0.000%	5/5	605.0 s
`kroB100`	3.696%	0.000%	0.000%	5/5	688.4 s
`ch150`	8.229%	0.000%	0.000%	5/5	603.9 s

The 10,000-kick campaign is deliberately expensive: mean wall time per seed is roughly 10–13 minutes under 20-worker parallel load. That cost is part of the result. A 20-kick table would have looked successful on three instances and hidden the `ch150` gap; the convergence campaign closes that gap and shows that the perturbation needs a longer plateau than the pilot suggested.

6.2 Broad panel and the LKH boundary

Across 78 TSPLIB instances with $n \leq 1000$, 236 of 390 runs stopped on patience and 183 hit the known optimum. The converged subset has median gap 0.000% and mean 0.100%. The large tail is weak: only 3 of 100 runs above $n = 300$ reached patience, and 97 hit the safety cap. The internal method is therefore a small/medium TSP solver, not a large-instance solver.

External LKH, at its stated budget and configuration, reaches 71 of 100 optima on the same large tail, mean gap 0.092%, median 0%, and worst 1.930%. LKH is optional external research-use software, not bundled and not claimed as Fresh Solutions' own heuristic. The comparison is capability-only, not matched-time.

The broad TSP panel also exposes the difference between a plateau stop and a safety-cap stop. A run that hits the 600-second cap is not evidence that the method converged; it is evidence that the method was still working when the budget ended. The table keeps those runs in the denominator and reports the converged subset separately. This is the same rule used in the QAP broad panel.

7 Applied case

TSA24/Prince George is a public-data applied case. The frozen build, target, and seed are published as `ws4-tsa24-evenflow-v1` in `https://github.com/UBC-FRESH/femic-tsa24-instance`, derived from public BC VRI, FADM, and SiteProd data. The even-flow target is a WS4 scenario configuration, not an official TSA decision.

The instance combines a compiled spatial model with an operations-research objective. The build freezes the block geometry, the operable-block set, the yield and treatment assumptions, and the candidate schedule structure. The target is a maximum-even-flow aspiration with a level term and a flow term; the objective is lower-is-better. The evidence records the compiled table identity, the target weights, the seed, and the schedule. A change to any of those invalidates the replay, which is the point of freezing the artifact rather than regenerating it from a mutable database.

The source data are public BC data with OGL-BC attribution. The compiled artifact is distributed under the terms recorded in the instance repository; the code and documentation in the papers repository are MIT and CC BY 4.0 as recorded in the licence files. The distinction matters because a public source dataset and a derived compiled model are not the same artifact.

The case has 208,255 operable blocks over a 200-year horizon. The even-flow target is a scenario configuration, not an official TSA decision. The 100,000-generation GPU-GA campaign spans

512.574–518.621 (mean 515.682) across five seeds. The seed-7 objective decomposes into a level term of 515.141 and a flow term of 2.721; the realized level is 51.3–52.4M m^3 per period against the 70M aspiration. That is a deliberate maximum-even-flow compromise, not a near-zero solution and not a feasibility failure.

The 1,000-generation record is 8,588,725.370498 and the 10,000-generation same-engine record is 76,540.740733; both replay exactly and show that the objective is still moving at short budgets. The CPU-SA independent-search baseline is 335,044,976.675435 after 1,875 seconds and is time-limited, not converged. The comparison is therefore a budget illustration, not a matched-performance claim.

The applied case is public-data and reproducible from the artifact, but it is not partner-endorsed and has no independent model baseline. It is included to show that the adapter and evidence boundary survive a compiled spatial model, not to claim an operational recommendation.

8 Discussion and limitations

8.1 What the harness buys

The harness makes several engineering properties explicit: the adapter boundary, exact resident state, replay, convergence records, and benchmark provenance. Those are useful even when the underlying heuristic is standard. A new problem can reuse the evidence machinery without pretending that a generic engine contains its best neighbourhood. A new comparison can reuse the replay gate without trusting a solver's internal objective counter. A reader can inspect a record, rerun a campaign, or disagree with a stopping rule without reverse-engineering an experimental script.

The strongest QAP and TSP results still come from adapter-specific modules (`qap_tabu.py` and `tsp_lk.py`). The harness does not turn those modules into a new algorithm, and the paper does not present them as state-of-the-art. The contribution is the boundary that makes their evidence comparable and replayable.

8.2 Framework use and overhead

The framework-use result is narrower than a generic search claim: a QAP tabu path held in the engine's resident state reproduces the private-array trajectory exactly on the tested panel. All 40 runs matched the standalone and production objectives, stopped on the same patience rule, and replayed exactly. The measured CPU overhead is 1.239 (95% bootstrap CI 1.168–1.310); per-instance means range from 1.170 to 1.327. This is a real cost, not a rounding error. It is also small enough to be acceptable when the alternative is rebuilding replay, provenance, and exact accounting around a new problem.

The overhead has a clear source. The standalone path mutates an array and updates the objective from the delta; the framework path routes the accepted swap through the adapter and calls the exact evaluator. The gap is the price of the evidence boundary. A user who needs the last few percent of single-run speed can keep a private array inside an adapter-specific module. A user who needs replayable evidence across problems can pay the measured cost and use the resident state.

8.3 Stopping rules and benchmark methodology

The QAP budget comparison is the clearest methodological result. The fixed 10,000-iteration table reported means of 0.000%, 0.403%, 0.685%, and 0.792%. The 100,000-patience panel reported 0.000%, 0.030%, 0.077%, and 0.245%. The difference is not a new algorithm; it is a longer plateau.

A paper that reports only the short budget would understate the method on three instances, while a paper that reports only the long budget would hide how much work the result required. The harness reports both.

The TSP result makes the same point in a different form. The 20-kick pilot reached three optima and a 0.383% mean gap on `ch150`. The 2,000-kick/200-patience campaign reached 19 of 20 optima. The 10,000-kick/1,000-patience campaign reached all 20. A short-budget table would have been publishable looking and wrong about the method's ceiling. The convergence table is longer and more expensive, but it is the honest one.

Patience is still a plateau rule, not global optimality. A deceptive landscape can be flat for a long time and then improve. The paper therefore says "plateau-stopped" or "converged for this evidence," not "proved optimal." The cap-hit counts are part of the result, not an appendix.

8.4 Limitations

The QAP convergence panel has four instances. It tests the framework path and the stopping rule, but it does not estimate performance across QAPLIB. The broad panel covers 134 parseable instances with a fixed budget; it is a screen, not a convergence study. The median gap of 0% on the broad panel is dominated by small instances, and the mean is pulled by hard outliers. The large-instance table is mixed: Fresh tabu wins some instances and loses `tai150b` to SciPy. The reference is robust tabu, not a state-of-the-art QAP method.

The TSP convergence panel has four instances. The 78-instance broad panel uses nearest-neighbour starts and a different budget; it is not the same experiment. The internal LK path does not scale to the largest TSPLIB tail, and the large-tail comparison against LKH is capability-only, not matched-time. LKH is external research-use software; it is not bundled and is not claimed as Fresh Solutions' own heuristic.

The applied case is public-data and reproducible from the artifact, but it is not partner-endorsed and has no independent model baseline. The CPU-SA baseline is time-limited, not converged. The applied section is an evidence-boundary demonstration, not an operational recommendation.

The generic simulated-annealing and population layers were tested for contract conformance and smoke behaviour, but they are not competitive with the adapter-specific references in the headline results. The framework-use experiment covers state management and replay, not a generic search engine. A reader who wants a general algorithm-selection framework should look elsewhere; a reader who wants a reproducible adapter and evidence layer is the intended user.

8.5 Operational implications

A team adopting the harness should freeze the instance before running the campaign. For a compiled spatial model, that means recording the table identity, target weights, and candidate set, not just the database version. For a public library instance, it means recording the file hash and known optimum. A result produced from a mutable instance is not comparable, even if the seed and solver version are recorded.

The second operational rule is to separate the search budget from the evidence budget. A short run can be useful for development, but it should not be promoted to a paper table without the stopping rule that produced it. The harness records the stop reason for this reason. A reviewer should be able to see whether a mean came from plateau-stopped runs or from a mixture of plateau stops and cap hits.

The third rule is to replay in continuous integration. A stored objective is not evidence until a fresh adapter evaluation reproduces it. The replay gate is cheap relative to the search and catches

serialization, permutation, and objective-accounting defects. It does not catch a bad search design, but it prevents a bad record from being reported as a good one.

The harness is not the right tool for every project. A team that already has a mature solver and a mature evidence pipeline should not adopt a new boundary for its own sake. A team that is rebuilding the same replay and provenance code for every problem is the intended user. The contribution is useful exactly where the engineering around the heuristic dominates the heuristic itself.

8.6 What would falsify the claim

The framework-use claim is narrow enough to falsify. It would fail if any seed produced a different framework objective from the standalone path, if a run hit the iteration cap, if a stored incumbent did not replay, or if the paired overhead were so large that the evidence boundary dominated the search. None of those failures occurred on the 40-run QAP panel: all objectives matched, all runs stopped on patience, all incumbents replayed, and the mean CPU ratio was 1.239 with an upper confidence bound of 1.310.

The claim would also fail if a new adapter required changes to the engine's search loop. The adapter for a new problem would then not be a boundary; it would be a patch. The current QAP and TSP adapters do not require such changes, but two adapters are not proof of generality. The applied spatial case is a third adapter in the programme, but it is not part of the QAP/TSP convergence panel and is not used to claim universal portability.

The TSP convergence claim would fail if the plateau-stopped runs were not reproducible, if the starts were not the stated Or-opt tours, or if the 10,000-kick run did not improve on the 2,000-kick run. The 2k pilot had a 0.077% worst gap on `ch150`; the 10k run reached the optimum on all five seeds. The improvement is evidence that the shorter budget was not converged, and the longer run is the one reported.

8.7 Practical guidance and claim boundary

Use the harness when the expensive part of the project is the model boundary: compiled spatial data, domain-specific feasibility, exact objective accounting, and evidence that must survive a code review. Do not use it as an excuse to skip algorithm engineering. The QAP and TSP modules are reference implementations; a new problem still needs a neighbourhood that fits its structure.

The evidence release and tag remain pending. No state-of-the-art or universal-dominance claim is made. The public artifact is a candidate until the maintainer approves the release. The paper is written to be useful before that approval, not to imply that an unreleased artifact is already archived.

8.8 Future work

The next evidence step is a wider convergence panel. The QAP side needs more sizes and a state-of-the-art baseline, not only robust tabu and SciPy. The TSP side needs more instances and a matched-time comparison against LKH, not only a capability boundary. The applied case needs an independent spatial baseline before it can support an operational claim.

The next framework step is a portability experiment with a third problem family and an adapter written by someone other than the reference author. The current QAP, TSP, and spatial adapters share the contract, but a third-party adapter is a stronger test of whether the boundary is real. A containerized artifact and a one-command replay are also needed before the evidence release can be called archival.

The next algorithmic step is optional. A generic tabu or large-neighbourhood layer could be added to the engine and tested against the adapter-specific modules, but that is a separate paper. The current paper deliberately stops at the evidence boundary. Adding a new heuristic to the engine would not strengthen the framework-use claim unless it were compared fairly and reproduced exactly.

9 Conclusion

Fresh Solutions provides a reproducible benchmark harness and adapter boundary, with two reference metaheuristic implementations, exact replay, and a bounded framework-use result for engine-managed QAP state. The QAP convergence panel shows that a 100,000-iteration plateau changes the reported gaps relative to a short fixed budget, while the framework path preserves the search trajectory at a measured 1.239 CPU ratio. The TSP 10,000-kick campaign closes the gap left by the 20-kick pilot and reaches all 20 known optima in the convergence panel.

The result is deliberately bounded. The panels are small, the QAP baseline is standard rather than state-of-the-art, the large TSP tail is weak, and external LKH is a capability reference rather than a component. The generic simulated-annealing layer is not presented as a competitive search method. The next step is not a larger claim; it is a wider convergence panel, stronger independent baselines, and an approved evidence release. The harness is useful now because it makes those next steps replayable.

A reader should be able to reproduce the QAP framework table, the TSP convergence table, and the replay counts from the committed runners and evidence without reconstructing the experimental protocol from prose. That is the standard the paper is trying to meet. It is also the standard by which the next version should be judged.

Data and code availability

QAPLIB and TSPLIB instances are public. The harness, reference implementations, runners, replay scripts, and evidence are in the Fresh Solutions papers repository. The framework-use evidence is `paper2/evidence/framework-use.json`; the TSP convergence evidence is `paper2/evidence/tsp-convergence-10k.json`; the shorter 2k and 20-kick runs are retained separately. The TSA24 artifact is available at `https://github.com/UBC-FRESH/femic-tsa24-instance/releases/tag/ws4-tsa24-evenflow-v1`. The evidence tag and release remain pending maintainer approval.

References

David L. Applegate, Robert E. Bixby, Vasek Chvátal, and William J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

Richard S. Barr, Bruce L. Golden, James P. Kelly, Mauricio G. C. Resende, and William R. Stewart. Designing and reporting computational experiments with heuristic methods. *INFORMS Journal on Computing*, 7(1):1–17, 1995.

Roberto Battiti and Giampietro Tecchiolli. The reactive tabu search. *ORSA Journal on Computing*, 6(2):126–140, 1994. doi: 10.1287/ijoc.6.2.126.

Vahid Beiranvand, Warren Hare, and Yves Lucet. Best practices for comparing optimization algorithms. *Optimization and Engineering*, 18(4):815–848, 2017. doi: 10.1007/s11081-017-9366-1.

Una Benlic and Jin-Kao Hao. Breakout local search for the quadratic assignment problem. *Applied Mathematics and Computation*, 219(9):4800–4815, 2013. doi: 10.1016/j.amc.2012.10.106.

Christian Blum and Andrea Roli. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys*, 35(3):268–308, 2003. doi: 10.1145/937503.937505.

Carl Boettiger. An introduction to docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71–79, 2015. doi: 10.1145/2723872.2723882.

Rainer E. Burkard, Stefan E. Karisch, and Franz Rendl. QAPLIB – a quadratic assignment problem library. *Journal of Global Optimization*, 10:391–403, 1997. doi: 10.1023/A:1008293323270.

David L. Donoho, Arian Maleki, Inam Ur Rahman, Morteza Shahram, and Victoria Stodden. Reproducible research in computational harmonic analysis. *Computing in Science & Engineering*, 11(1):8–18, 2009. doi: 10.1109/MCSE.2009.15.

Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. COCO: a platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36(1):114–144, 2020. doi: 10.1080/10556788.2020.1808977.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

Keld Helsgaun. An effective implementation of the lin–kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000. doi: 10.1016/S0377-2217(99)00284-2.

Keld Helsgaun. LKH-3: an extension of the lin–kernighan–helsgaun TSP solver. `https://github.com/cerebis/LKH3`, 2019. External research-use software; not bundled.

J. N. Hooker. Testing heuristics: We have it all wrong. *Journal of Heuristics*, 1(1):33–42, 1995. doi: 10.1007/BF02430364.

Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations and Applications*. Morgan Kaufmann, 2004.

David S. Johnson and Lyle A. McGeoch. The traveling salesman problem: A case study in local optimization. In Emile Aarts and Jan Karel Lenstra, editors, *Local Search in Combinatorial Optimization*, pages 215–310. Wiley, 1997.

Tjalling C. Koopmans and Martin Beckmann. Assignment problems and the location of economic activities. *Econometrica*, 25(1):53–76, 1957.

Gregory M. Kurtzer, Vanessa Sochat, and Michael W. Bauer. Singularity: Scientific containers for mobility of compute. *PLoS ONE*, 12(5):e0177459, 2017. doi: 10.1371/journal.pone.0177459.

Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: a LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015. doi: 10.1145/2833157.2833162.

Shen Lin and Brian W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973. doi: 10.1287/opre.21.2.498.

Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter Hahn, and Tania Querido. A survey for the quadratic assignment problem. *European Journal of Operational Research*, 176(2):657–690, 2007. doi: 10.1016/j.ejor.2005.09.032.

Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Birattari, and Thomas Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016. doi: 10.1016/j.orp.2016.09.002.

Helena R. Lourenço, Olivier C. Martin, and Thomas Stützle. Iterated local search. In Fred Glover and Gary Kochenberger, editors, *Handbook of Metaheuristics*, pages 320–353. Kluwer, 2003.

Catherine C. McGeoch. Toward an experimental method for algorithm simulation. *INFORMS Journal on Computing*, 8(1):1–15, 1996. doi: 10.1287/ijoc.8.1.1.

Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.

Gerhard Reinelt. TSPLIB – a traveling salesman problem library. *ORSA Journal on Computing*, 3 (4):376–384, 1991. doi: 10.1287/ijoc.3.4.376.

Sartaj Sahni and Teofilo Gonzalez. P-complete approximation problems. *Journal of the ACM*, 23 (3):555–565, 1976.

Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.

Victoria Stodden, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P. A. Ioannidis, and Michela Taufer. Enhancing reproducibility for computational methods. *Science*, 354(6317):1240–1241, 2016. doi: 10.1126/science.aah6168.

Éric D. Taillard. Robust taboo search for the quadratic assignment problem. *Parallel Computing*, 17(4–5):443–455, 1991. doi: 10.1016/S0167-8191(05)80147-4.

Éric D. Taillard. Comparison of iterative searches for the quadratic assignment problem. *Location Science*, 3(2):87–105, 1995. doi: 10.1016/0966-8349(95)00008-6.

UBC FRESH Lab. TSA24 even-flow WS4 benchmark artifact, version 1. `https://github.com/UBC-FRESH/femic-tsa24-instance/releases/tag/ws4-tsa24-evenflow-v1`, 2026.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

Greg Wilson, D. A. Aruliah, C. Titus Brown, Neil P. Chue Hong, Matt Davis, Richard T. Guy, Steven H. D. Haddock, Kathryn D. Huff, Ian M. Mitchell, Mark D. Plumbley, Ben Waugh, Ethan P. White, and Paul Wilson. Best practices for scientific computing. *PLoS Biology*, 12(1): e1001745, 2014. doi: 10.1371/journal.pbio.1001745.